Mithril in Game-based Syntax

Wuyunsiqin Bingsheng Zhang
Zhejiang University
oyun@zju.edu.cn bingsheng@zju.edu.cn

November 25, 2025

1 Preliminaries

1.1 Notations

There are some basic notation and conventions used throughout this work.

- The security parameter is denoted by $\lambda \in \mathbb{N}$.
- For any finite set S, we write $|S|$ for its cardinality. We use the notation

$$x \xleftarrow{\$} S$$

  to indicate that x is sampled uniformly at random from the finite set S.
- A function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is called *negligible* if for every polynomial $p(\cdot)$ there exists $N \in \mathbb{N}$ such that for all $\lambda \geq N$ it holds that $\mathsf{negl}(\lambda) < 1/p(\lambda)$. We write $\mathsf{negl}(\lambda)$ to denote an unspecified negligible function in the security parameter λ.

1.2 BLS Signature Scheme

Let λ be the security parameter. We assume a bilinear group generation algorithm which, on input 1^λ, outputs

$$(q, G_1, G_2, G_T, g_1, g_2, e),$$

where G_1, G_2, G_T are cyclic groups of prime order q, $g_1 \in G_1$, $g_2 \in G_2$ are generators, and $e : G_1 \times G_2 \to G_T$ is a bilinear pairing. Let $H : \{0,1\}^* \to G_1$ be a hash function modeled as a random oracle.

We write the BLS signature scheme as

$$\Pi_{\mathsf{BLS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Sign}, \mathsf{Ver}).$$

- $pp \leftarrow \mathsf{Setup}(1^\lambda)$: run the group generator to obtain $(q, G_1, G_2, G_T, g_1, g_2, e)$ and fix H. Output public parameters

$$\mathsf{pp} = (q, G_1, G_2, G_T, g_1, g_2, e, H).$$

- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$: sample a secret key $sk \xleftarrow{\$} \mathbb{Z}_q$, where $\mathbb{Z}_q$ denotes the integers modulo q, and set

$$vk = g_2^{sk} \in G_2.$$

  Output the key pair (sk, vk).

- $\sigma \leftarrow \mathsf{Sign}(\mathsf{pp}, sk, msg)$: on input a message $msg \in \{0,1\}^*$, compute $h \leftarrow H(msg) \in G_1$ and set

$$\sigma = h^{sk} \in G_1.$$

  Output the signature σ.

- $b \leftarrow \mathsf{Ver}(\mathsf{pp}, vk, msg, \sigma)$, where $b \in \{0,1\}$: compute $h \leftarrow H(msg)$ and output

$$b = 1 \text{ iff } e(\sigma, g_2) = e(h, vk).$$

1.3 Merkle Tree

Let $H_{\mathsf{MT}} : \{0,1\}^* \to \{0,1\}^\lambda$ be a collision-resistant hash function.

We denote the Merkle tree primitive by

$$\Pi_{\mathsf{MT}} = (\mathsf{Create}, \mathsf{Check}).$$

- $\mathsf{avk} \leftarrow \mathsf{Create}(\mathbf{vk}, \mathbf{w})$: on input a vector of verification keys $\mathbf{vk} = (vk_1, \ldots, vk_N)$ and a matching stake vector $\mathbf{w} = (w_1, \ldots, w_N)$, where $N \in \mathbb{N}$ denotes the number of parties, set leaves $\ell_i := H_{\mathsf{MT}}(vk_i, w_i)$ and output the Merkle root avk, which we also call an aggregate verification key.

- $b \leftarrow \mathsf{Check}(\mathsf{avk}, vk, w, path)$, where $b \in \{0,1\}$: on input a root avk, a pair (vk, w), and an authentication path $path$, recompute the root from the leaf $\ell := H_{\mathsf{MT}}(vk, w)$ and $path$. Output $b = 1$ iff the recomputed root equals avk, and $b = 0$ otherwise.

2 Stake-based threshold multi-signature

2.1 Syntax

We consider a stake-based threshold multi-signature (STMS) scheme whose acceptance is determined on a per-stake basis via a "t wins in n lotteries" parametrization, for some integers $t, n \in \mathbb{N}$.

$$\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer}).$$

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$: on input a security parameter 1^λ, outputs public system parameters pp. These parameters are implicit inputs to all other algorithms.

- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$: on input the public parameters pp, generates a signing/verification key pair (sk, vk) for a single party.

- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$: given a list of verification keys $\mathbf{vk} := (vk_1, \ldots, vk_N)$ and a matching stake vector $\mathbf{w} := (w_1, \ldots, w_N)$, where $N \in \mathbb{N}$ denotes the number of parties, deterministically produces an aggregate verification key avk.

- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0, 1\}$: deterministically checks whether avk is a valid aggregate verification key for $(\mathbf{vk}, \mathbf{w})$, outputting $b = 1$ if it accepts and $b = 0$ otherwise.

- $\sigma \leftarrow \mathsf{Sig}(vk, w, sk, j, m)$: attempts to produce a signature σ for message m and lottery index $j \in [n]$ under verification key vk, secret key sk, and stake w. It outputs $\perp$ if the user is ineligible for (m, j) under stake w.

- $b \leftarrow \mathsf{Ver}(vk, w, j, m, \sigma)$, where $b \in \{0, 1\}$: given a verification key vk, a stake value w, a lottery index j, a message m, and a signature σ, deterministically checks whether σ is a valid signature for (m, j) under (vk, w) and outputs $b = 1$ if it accepts and $b = 0$ otherwise.

- $\tau \leftarrow \mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{(vk_i, \sigma_i)\}_{i=1}^t)$: given $(\mathbf{vk}, \mathbf{w})$, a message m, and a multiset $\{(vk_i, \sigma_i)\}_{i=1}^t$ of signatures, attempts to produce an aggregate signature τ; it outputs $\perp$ if the provided inputs are insufficient or inconsistent.

- $b \leftarrow \mathsf{AVer}(\mathsf{avk}, m, \tau, T)$, where $b \in \{0, 1\}$: given a message m, an aggregate verification key avk, an aggregate signature τ, and a stake threshold parameter T, outputs $b = 1$ if τ is accepted as a valid aggregate (stake-based threshold) multi-signature on m under avk and threshold T, and $b = 0$ otherwise.

2.1.1 Correctness

Let

$$\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$$

be a stake-based threshold multi-signature (STMS) scheme as specified above. Let $\mathcal{M}$ denote the message space.

Definition (Correctness). We say that Π_{STMS} is *correct* if for every security parameter $\lambda \in \mathbb{N}$, every number of parties $N \in \mathbb{N}$, every vector of verification keys $\mathbf{vk} = (vk_1, \ldots, vk_N)$ generated honestly by KGen, every stake vector $\mathbf{w} = (w_1, \ldots, w_N)$, every message $m \in \mathcal{M}$, every stake threshold parameter T, and every lottery index j in the relevant index set, the following properties hold

except with negligible probability in λ over the randomness of all algorithms involved.

- **Correctness of aggregate key.** If $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$, then
$$\mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w}) = 1.$$
- **Correctness of individual signatures.** For every party index $i \in [N]$, let (sk_i, vk_i) be generated by $(sk_i, vk_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$. If $\sigma_i \leftarrow \mathsf{Sig}(vk_i, w_i, sk_i, j, m)$ and $\sigma_i \neq \perp$, then
$$\mathsf{Ver}(vk_i, w_i, j, m, \sigma_i) = 1.$$
- **Correctness of aggregate signatures.** Let $t \in \mathbb{N}$ and, for each $i \in [t]$, let (sk_i, vk_i) be generated by $(sk_i, vk_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$ and let w_i be the corresponding stake value. For a message m and a lottery index j, let $\sigma_i \leftarrow \mathsf{Sig}(vk_i, w_i, sk_i, j, m)$ be such that $\sigma_i \neq \perp$ for all $i \in [t]$. Let
$$\tau \leftarrow \mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{(vk_i, \sigma_i)\}_{i=1}^{t}) \quad \text{and} \quad \mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w}).$$
If the total stake of these t parties meets the threshold, i.e.,
$$\sum_{i=1}^{t} w_i \;\geq\; T,$$
then
$$\mathsf{AVer}(\mathsf{avk}, m, \tau, T) = 1.$$

2.1.2 Security

Stake-based unforgeability α-UF-CMA. In the unforgeability experiment, the challenger runs Setup, generates honest keys (sk_i, vk_i), fixes $\mathbf{w}$, computes avk, and gives $(\mathsf{pp}, \mathbf{vk}, \mathbf{w}, \mathsf{avk})$ to the adversary $\mathcal{A}$. The adversary can:

- query *signing oracles* for any honest user i, any j, and any message m, receiving $\mathsf{Sig}(vk_i, w_i, sk_i, j, m)$;
- *corrupt* users to obtain sk_i (and then act under those keys).

Let C^* be the set of corrupted users. Let H^* be the set of honest users that *participated* on the forged topic (made at least one signing query for the challenge topic) under the forged aggregate key. The experiment *charges* only successful eligibility (i.e., $\mathsf{Sig} \neq \perp$) and enforces the stake bound
$$\sum_{i \in C^* \cup H^*} w_i \;\leq\; \alpha W.$$
Finally, $\mathcal{A}$ outputs $(\mathbf{vk}^*, \mathsf{avk}^*, m^*, \tau^*)$; it wins if
$$\mathsf{ACheck}(\mathsf{avk}^*, \mathbf{vk}^*, \mathbf{w}) = 1, \quad \mathsf{AVer}(\mathsf{avk}^*, m^*, \tau^*, W) = 1,$$
and τ^* is *fresh*, i.e., it cannot be obtained by aggregating local proofs returned (or derivable) from oracle responses. An STMS is α-UF-CMA secure if every PPT adversary wins with only negligible probability.

Strengthened unforgeability α-UF-CMA-S. As above, but the adversary may first register arbitrary public keys (and corresponding stake placement) before AKey is run (key-substitution/registration stage). The rest of the game is identical. Security requires negligible winning probability for all PPT adversaries.

2.2 Instantiation

2.2.1 Mithril

We write the Mithril scheme as

$$\Pi_{\mathsf{Mithril}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{ASig}, \mathsf{AVer}).$$

- $(\mathsf{pp}) \leftarrow \mathsf{Setup}(1^\lambda)$:
  run $\Pi_{\mathsf{BLS}}.\mathsf{Setup}(1^\lambda)$ to obtain $(q, G_1, G_2, G_T, g_1, g_2, e, H)$
  fix a dense mapping / eligibility function
  $$\mathsf{Eval} : (msg, index, \sigma) \mapsto \{0,1\}^\lambda,$$
  a weighting function
  $$\varphi : \mathbb{N} \to [0,1],$$
  e.g. $\varphi(w) = 1 - (1-f)^w$.
  Index range
  $$\mu \in \mathbb{N},$$
  a quorum or lottery parameter
  $$k \in \{1, \ldots, \mu\}.$$
  Output
  $$\mathsf{pp} = (q, G_1, G_2, G_T, g_1, g_2, e, H, \mathsf{Eval}, \varphi, \mu, k).$$
- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$:
  parse pp_{BLS} for BLS signature, run $\Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$
  $$(sk, vk) \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$$
  Output (sk, vk).
- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$:
  given
  $$\mathbf{vk} = (vk_1, \ldots, vk_N),$$
  $$\mathbf{w} = (w_1, \ldots, w_N),$$
  set
  $$\mathsf{avk} \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$

- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0,1\}$:
  compute
  $$avk' \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$
  output b=1 iff avk=avk'.

- $\pi \leftarrow \mathsf{Sig}(\mathsf{pp}, vk, w, sk, j, m)$:
  compute the BLS signature
  $$\sigma \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{Sign}(\mathsf{pp}, sk, m),$$
  compute the eligibility value
  $$ev \leftarrow \mathsf{Eval}(\sigma, j, m),$$
  and check the threshold
  $$ev \;<\; \lfloor \varphi(w) \cdot 2^{\lambda} \rfloor.$$
  If true, output
  $$\pi = (j, \sigma, vk, w).$$
  Otherwise output $\perp$.

- $\tau \leftarrow \mathsf{ASig}(\mathsf{pp}, m, \{\pi_i\}_{i=1}^{t})$:
  parse each
  $$\pi_i = (j_i, \sigma_i, vk_i, w_i),$$
  with pairwise distinct
  $$j_i \in \{1, \ldots, \mu\}.$$
  Set
  $$\sigma^{\star} \;=\; \prod_{i=1}^{t} \sigma_i \;\in\; G_1,$$
  $$vk^{\star} \;=\; \prod_{i=1}^{t} vk_i \;\in\; G_2.$$
  Derive a batching seed
  $$e_{\sigma} \;\leftarrow\; H\big(\sigma_1 \,\|\, \cdots \,\|\, \sigma_t\big),$$
  and coefficients for $i = 1, \ldots, t$
  $$\rho_i \;\leftarrow\; \mathsf{H}_{\lambda}(e_{\sigma} \,\|\, i) \in \mathbb{Z}_q^{*}.$$
  Compute the batch aggregates
  $$\mu \;=\; \prod_{i=1}^{t} \sigma_i^{\rho_i} \;\in\; G_1,$$

$$ivk \;=\; \prod_{i=1}^{t} vk_i^{\rho_i} \;\in\; G_2.$$

Output

$$\tau \;=\; (\,\sigma^\star,\, vk^\star,\, \mu,\, ivk,\, e_\sigma).$$

- $b \leftarrow \mathsf{AVer}(\mathsf{pp}, \mathsf{avk}, m, \tau)$, where $b \in \{0,1\}$:
  parse

  $$\tau = (\sigma^\star, vk^\star, \mu, ivk, e_\sigma).$$

  Check that the indices are pairwise distinct. Compute

  $$h \leftarrow H(m).$$

  Verify the equation

  $$e(\sigma^\star, g_2) \;=\; e(h, vk^\star).$$

  Recompute the batching coefficients for $i = 1, \ldots, t$

  $$\rho_i \;\leftarrow\; \mathsf{H}(e_\sigma \,\|\, i) \in \mathbb{Z}_q^*,$$

  and the corresponding aggregated public key

  $$ivk' \;=\; \prod_{i=1}^{t} vk_i^{\rho_i} \;\in\; G_2.$$

  Check consistency of the submitted aggregated key

  $$ivk' \;=\; ivk.$$

  Verify the batched equation

  $$e(\mu, g_2) \;=\; e(h, ivk).$$

  Output

  $$b \;=\; 1 \text{ iff all checks above hold.}$$